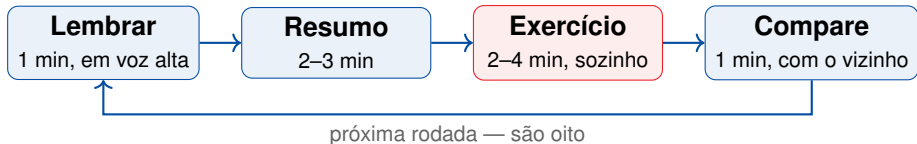


# Revisão de Programação C

Oito rodadas de resumo e exercício rápido

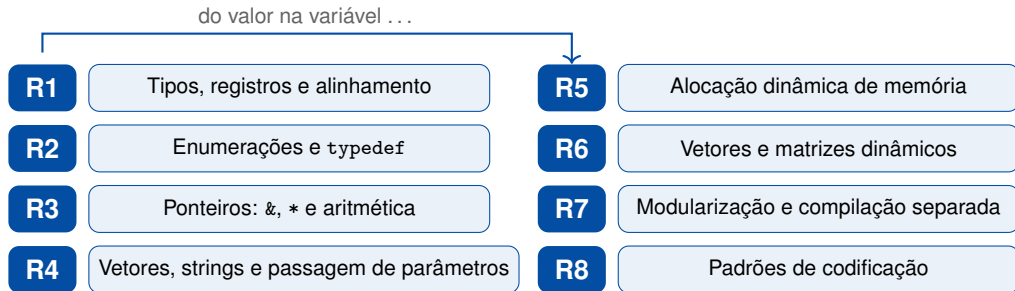
Prof. André Yoshiaki Kashiwabara

DCOMP – Universidade Federal de Sergipe  
[andreyoshiaki@dcomp.ufs.br](mailto:andreyoshiaki@dcomp.ufs.br)



## Combinado

- **Tudo no papel, sem computador:** numere uma folha de **R1** a **R8** e responda lendo o código, *antes* de conversar com o vizinho.
- Ninguém corrige no meio: o gabarito vem todo junto no final, e cada um confere a própria folha.
- Errou uma rodada? O último slide diz qual aula e quais exercícios da lista revisar.



... ao programa em vários arquivos, dentro de um padrão

# R1 — Tipos e registros

## Antes do resumo: responda em voz alta (1 min)

- Quanto vale  $7 / 2$  em C? E  $7 / 2.0$ ?
- O que acontece com um registro quando ele é passado para uma função?
- Por que `sizeof` de um registro pode ser maior que a soma dos campos?

Rodada 1 — aulas de origem: fundamentos de C; tipos, estruturas e TADs; registros e alinhamento

## Tipo de dados

Diz como ler os bits e quais operações valem. `int / int` é divisão **inteira**: `7/2 == 3`. Basta um operando `double` para a conta ser real.

## Registro (struct)

Agrupar campos de tipos diferentes sob um nome; acesso com ponto. Atribuição, passagem e retorno são por **cópia** — do registro inteiro.

## Tipo abstrato de dados

Separa *o que* se pode fazer (operações) de *como* é feito (a estrutura por dentro). Quem usa só conhece a interface.

## Vetores paralelos × vetor de registros

`nome[i], nota[i] ...` viram  
`turma[i].nome, turma[i].nota`: os dados de um aluno ficam juntos.

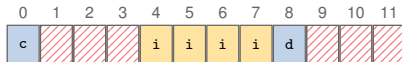
Fonte: Kernighan & Ritchie (1989), §2.2, §2.5 e cap. 6 [3]

**As três regras**, na ordem dos campos:

1. cada campo tem alinhamento  $a$  (char 1, int 4, double 8);
2. o campo começa no próximo offset múltiplo de  $a$  — os bytes pulados são **padding**;
3. o tamanho total sobe até múltiplo do **maior  $a$** .

Ordenar do maior para o menor alinhamento reduz o desperdício.

```
struct { char c; int i; char d; }
```



sizeof = 12, com 6 de padding

reordenado (int, char, char): 8

Fonte: Bryant & O'Hallaron (2015), §3.9.3 [1]

## Exercício R1 — 3 min, no papel



```
1 struct Leitura {  
2     char    sensor;  
3     double  valor;  
4     int     codigo;  
5 };
```

Considere uma máquina de 64 bits comum (char 1, int 4, double 8).

### Responda na folha

- (a) Em que offset começa cada campo?
- (b) Quanto vale `sizeof(struct Leitura)`?
- (c) Reordene os campos para gastar o mínimo. Qual o novo `sizeof`?

Dica: desenhe a fita de bytes, como no slide anterior.

**R2** — enum **e** typedef

## Antes do resumo: responda em voz alta (1 min)

- Sem valores explícitos, que número recebe a primeira constante de um `enum`?
- Uma variável `enum` aceita só os valores listados?
- `typedef` cria um tipo novo?

Rodada 2 — aulas de origem: enumerações e `typedef`

```
typedef enum {  
    CURSANDO, APROVADO, REPROVADO,  
    NUM_SITUACOES    /* vale 3 */  
} Situacao;  
int total[NUM_SITUACOES] = {0};  
  
switch (s) {  
case CURSANDO: ...  
case APROVADO: ...  
case REPROVADO: ...  
}
```

- enum nomeia inteiros: começa em 0, e cada constante sem valor vale a anterior +1.
- Em switch, -Wall avisa o caso esquecido.
- A sentinela final conta os valores — se ninguém atribuir valores à mão.
- Não é trava: a variável aceita qualquer inteiro.
- typedef é só apelido; em vetor, esconde que ele não se copia com =.

Fonte: Kernighan & Ritchie (1989), §2.3 e §6.7 [3]; King (2008), §16.5 [4]

```
1 enum Cor { VERMELHO, VERDE = 5,  
2           AZUL, NUM_CORES };  
3  
4 enum Cor c = 42;  
5  
6 typedef char Nome[20];  
7 Nome a = "Ana", b;  
8 b = a;
```

### Responda na folha

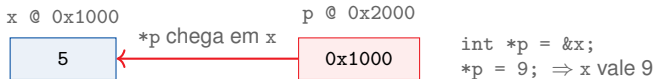
- (a) Quanto valem AZUL e NUM\_CORES?  
NUM\_CORES ainda conta quantas cores existem?
- (b) A linha 4 compila?
- (c) A linha 8 compila?

# R3 — Ponteiros

## Antes do resumo: responda em voz alta (1 min)

- O que um ponteiro guarda?
- Qual a diferença entre `&x` e `*p`?
- Se `p` aponta para um `int`, quantos bytes `p + 1` avança?

Rodada 3 — aulas de origem: ponteiros: conceitos, operadores e aritmética



## Os dois operadores

`&x`: o endereço de `x`.

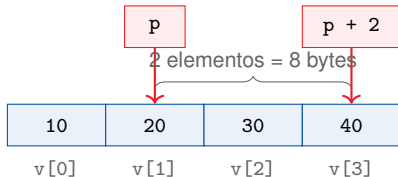
`*p`: o conteúdo no endereço guardado em `p`.

São inversos: `*(&x)` é `x`.

## Por que existem

C passa tudo por valor. Para uma função alterar `x`, passe `&x` e use `*` lá dentro — é o que `scanf` e `troca` fazem.

Fonte: Kernighan & Ritchie (1989), §5.1–5.2 [3]



- $p + k$  avança  $k$  elementos:  $k \times \text{sizeof}(*p)$  bytes.
- $v[i]$  é exatamente  $*(v + i)$ ; o nome do vetor vale o endereço de  $v[0]$ .
- $p - q$  (mesmo vetor) dá a distância em elementos, do tipo `ptrdiff_t`.

Fonte: Kernighan & Ritchie (1989), §5.3–5.4 [3]

## Exercício R3 — 3 min, no papel



```
1  int v[] = { 10, 20, 30, 40 };
2  int *p = v + 1;
3  int x = 5;
4  int *q = &x;
5
6  *q = *p + 1;
7  p += 2;
8  *p = x * 2;
9
10 printf("%d %d %d %td\n",
11         x, v[3], *(v + 2), p - v);
```

### Responda na folha

- (a) Para onde *p* aponta depois da linha 7?
- (b) O que o programa imprime?

Dica: desenhe o vetor e mova a seta de *p* a cada linha.

## **R4 — Vetores e strings**

## Antes do resumo: responda em voz alta (1 min)

- O que uma função recebe quando você passa um vetor?
- Qual a diferença entre `sizeof(s)` e `strlen(s)`?
- Quando o `&` é necessário na chamada?

Rodada 4 — aulas de origem: vetores, strings e passagem por referência

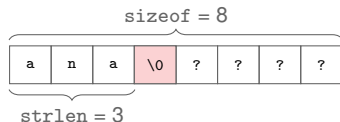
## Vetor em função

O vetor **decai** para o endereço do primeiro elemento: `int v[]`, `int v[10]` e `int *v` são o mesmo parâmetro. O tamanho não viaja: vai como parâmetro extra.

## C só passa por valor

Escalar que deve mudar: `&` na chamada, `*` no uso. Vetor e string já chegam como endereço.

```
char s[8] = "ana";
```



String é vetor de `char` com a marca `'\0'` no fim do conteúdo. Comparar é com `strcmp`, nunca com `==`.

Fonte: Kernighan & Ritchie (1989), §1.9, §5.3 e §5.5 [3]

```
1 void f(int v[10]) {  
2     printf("B: %zu\n", sizeof(v));  
3 }  
4  
5 int main(void) {  
6     int a[10];  
7     char s[20] = "prova";  
8     printf("A: %zu %zu %zu\n",  
9         sizeof(a), sizeof(s), strlen(s));  
10    f(a);  
11    s[2] = '\0';  
12    printf("C: %s %zu\n", s, strlen(s));  
13 }
```

## Responda na folha

O que aparece nas linhas A:, B: e C:?

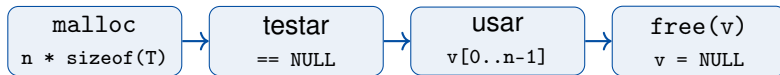
Máquina de 64 bits: int 4, ponteiro 8.

# **R5 — Alocação dinâmica**

## Antes do resumo: responda em voz alta (1 min)

- Onde vive um bloco de `malloc`, e até quando?
- O que `malloc` devolve quando não há memória?
- Por que `v = realloc(v, ...)` é perigoso?

Rodada 5 — aulas de origem: alocação dinâmica de memória



- O ponteiro vive na pilha; o bloco vive na **heap** até o `free`, mesmo que o ponteiro se perca.
- `malloc`: conteúdo indeterminado.  
`calloc(q, t)`: zerado.
- `realloc` pode mudar o endereço e devolve `NULL` na falha: receba em um **temporário**.
- `free` devolve o bloco mas não mexe no ponteiro.

Fonte: Kernighan & Ritchie (1989), §7.8.5 [3]; King (2008), §17.1–17.4 [4]

| Falha                   | O que é                          | Sintoma típico                          |
|-------------------------|----------------------------------|-----------------------------------------|
| Vazamento               | alocar e nunca liberar           | programa incha com o tempo              |
| Ponteiro pendurado      | usar depois do <code>free</code> | lixo ou <i>segfault</i> aleatório       |
| <code>free</code> duplo | liberar duas vezes               | aborta em <code>free</code> , sem pista |
| Estouro do bloco        | escrever além do tamanho         | corrompe o vizinho                      |

## Nenhuma falha onde foi cometida

O programa quebra depois, em outro lugar. Quem aponta a linha certa é a ferramenta:  
`gcc -g -fsanitize=address.`

Fonte: Serebryany et al. (2012) [6]; King (2008), §17.4 [4]

```
1  int *cria(int n) {
2      int *v = malloc(n);
3      for (int i = 0; i < n; i++)
4          v[i] = i * i;
5      return v;
6  }
7
8  int main(void) {
9      int *v = cria(5);
10     v = realloc(v, 10 * sizeof(int));
11     free(v);
12     printf("%d\n", v[4]);
13     return 0;
14 }
```

### Responda na folha

O compilador aceita este código sem nenhum aviso — e ele tem **quatro** problemas. Para cada um: a linha, qual é o problema e como corrigir.

Use a tabela das quatro falhas e o ciclo pedir–verificar–usar–devolver.

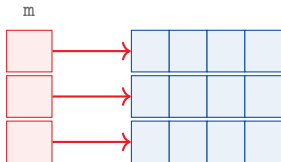
## **R6 — Matrizes dinâmicas**

## Antes do resumo: responda em voz alta (1 min)

- Quais são as duas formas de alocar uma matriz  $n_l \times n_c$ ?
- Em que ordem se libera uma `double **`?
- No bloco único, onde fica o elemento  $(i, j)$ ?

Rodada 6 — aulas de origem: alocação dinâmica de vetores e matrizes

## Vetor de ponteiros (double \*\*)



- $1 + nl$  malloc;  $m[i][j]$
- falha no meio: desfazer o que já deu certo
- liberar de **dentro para fora**: linhas, depois  $m$

## Bloco único (double \*)



- um malloc, um free; linhas contíguas
- elemento  $(i, j)$  em  $m[i * nc + j]$ ;  $nc$  viaja junto
- $(size\_t)nl * nc$  evita estouro de int

Fonte: Kernighan & Ritchie (1989), §5.7–5.9 [3]

Uma matriz com  $n_l = 3$  linhas e  $n_c = 4$  colunas.

```
1  /* liberando a versao double ** */
2  free(m);
3  for (int i = 0; i < n_l; i++)
4      free(m[i]);
```

### Responda na folha

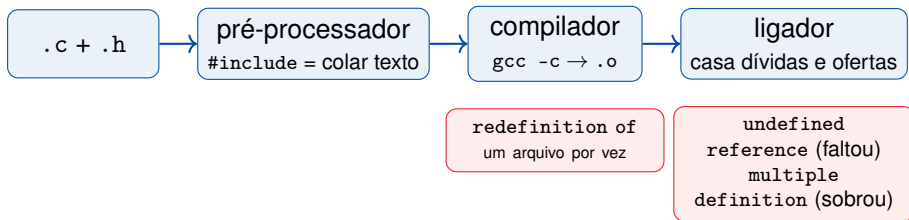
- (a) No bloco único, em que índice fica o elemento da linha 2, coluna 1?
- (b) Quantas chamadas a `malloc` cada versão faz?
- (c) O que está errado na liberação ao lado? Corrija.

# R7 — Modularização

## Antes do resumo: responda em voz alta (1 min)

- O que vai no .h e o que fica no .c?
- Para que serve a guarda #ifndef/#define/#endif?
- undefined reference é erro do compilador ou do ligador?

Rodada 7 — aulas de origem: modularização: headers, múltiplos arquivos e bibliotecas



- O `.h` é o contrato: protótipos, tipos, constantes — nunca corpos de função nem definição de variáveis.
- Todo `.h` começa com `#ifndef X_H / #define X_H` e termina com `#endif`.
- `static` numa função a esconde do ligador: dois `.c` podem ter a sua.

Fonte: Kernighan & Ritchie (1989), §4.5, §4.6 e §4.11.3 [3]; King (2008), cap. 15 [4]

# Exercício R7 — 4 min, no papel



```
/* estat.h (sem guarda) */
struct Resumo {
    double media, maior;
};

double estat_media(
    const double *v, int n);

/* turma.h (sem guarda) */
#include "estat.h"
struct Resumo turma_resume(
    const double *v, int n);
```

```
/* main.c */
#include "estat.h"
#include "turma.h"
int main(void) { ... }

/* estat.c e turma.c:
cada um define a sua
void troca(double *a,
            double *b)
{ ... } */
```

## Responda na folha

Um colega digitou estes comandos, nesta ordem. Sem executar: dá erro? Em qual etapa, de que tipo, e como corrigir?

- (a) gcc -c main.c
- (b) com (a) corrigido:  
gcc main.c turma.c
- (c) gcc main.c turma.c  
estat.c

# R8 — Padrões de codificação

## Antes do resumo: responda em voz alta (1 min)

- Por que evitar variáveis globais?
- O que é um número mágico?
- O que `static` e `const` dizem a quem lê o código?

Rodada 8 — aulas de origem: padrões de codificação e prática de modularização

- ☐ todo nome público começa com o prefixo do módulo (`ranking_`)?
- ☐ nome de uma letra só dentro de laço curto?
- ☐ nenhuma variável global — o estado viaja por parâmetro?
- ☐ nenhum número solto: limites e limiares têm nome?
- ☐ tudo que não está no `.h` é `static`?
- ☐ `const` em todo parâmetro que a função só lê?
- ☐ cada função do `.h` diz o que devolve e quem libera a memória?
- ☐ uma formatação só (`clang-format`) e `-Wall -Wextra` limpo?

Funcionar não é o mesmo que estar pronto: o padrão existe para o próximo leitor — que muitas vezes é você.

Fonte: McConnell (2004), caps. 11 e 32 [5]; Kernighan & Pike (1999), cap. 1 [2]

```
1  /* ranking.c */
2  int n, k;
3  double R[50];
4
5  void calc(double *v)
6  {
7      k = 0;
8      for (int i = 0; i < n; i++)
9          if (v[i] >= 7.0)
10             R[k++] = v[i];
11 }
```

### Responda na folha

- (a) Aponte **quatro** problemas usando o checklist.
- (b) O que acontece se a turma tiver 60 alunos aprovados?
- (c) Reescreva só a **assinatura** de `calc` dentro do padrão, sem globais.

**Gabarito**

## Como conferir

Marque na folha: **C** (certo), **P** (parcial) ou **E** (errado) em cada rodada. O placar diz o que revisar antes da prova — está no último slide.



- (a) sensor em 0; valor em **8** (múltiplo de 8, pula 7 bytes); código em **16**.
- (b) Ocupa 20 bytes, mas sobe até múltiplo de 8: `sizeof` = **24** (11 de padding).
- (c) `double valor; int código; char sensor;`  $\Rightarrow 8 + 4 + 1 = 13$ , arredondado para **16**.

Fonte: Bryant & O'Hallaron (2015), §3.9.3 [1]

- (a) VERMELHO = 0, VERDE = 5, AZUL = 6, NUM\_CORES = 7. São 3 cores, mas a sentinela vale 7: o truque do contador só funciona sem valores explícitos. `int total[NUM_CORES]` desperdiçaria 4 posições.
- (b) **Compila**, sem aviso, e `c` vale 42. O `enum` dá nome a valores, mas não restringe a variável.
- (c) **Não compila**. Nome é um vetor de 20 `char`, e vetor não se atribui — o `typedef` só escondeu isso. Para copiar: `strcpy(b, a);`

```
error: assignment to expression with array type
```

Fonte: Kernighan & Ritchie (1989), §2.3 e §6.7 [3]

| linha | efeito               | estado      |
|-------|----------------------|-------------|
| 2     | $p \rightarrow v[1]$ |             |
| 4     | $q \rightarrow x$    |             |
| 6     | $x = v[1] + 1$       | $x = 21$    |
| 7     | $p \rightarrow v[3]$ |             |
| 8     | $v[3] = 21 * 2$      | $v[3] = 42$ |

## Respostas

(a)  $p$  aponta para  $v[3]$ : avançou 2 *elementos* a partir de  $v[1]$ .

(b) Imprime:

21 42 30 3

$*(v+2)$  é  $v[2]$ , intocado;  $p - v$  é a distância em elementos.

Fonte: Kernighan & Ritchie (1989), §5.4 [3]

```
A: 40 20 5  
B: 8  
C: pr 2
```

- A: `a` tem `10 int = 40 bytes`; `s` tem capacidade 20; o conteúdo "prova" tem 5.
- B: dentro de `f`, `v` é um `int *`: `sizeof` dá o tamanho do **ponteiro**, 8 — o `[10]` da declaração é ignorado. O próprio compilador avisa: `-Wsizeof-array-argument`.
- C: o `'\0'` na posição 2 corta a string: "pr", tamanho 2. Os bytes ova continuam lá, mas ninguém os lê.

Fonte: Kernighan & Ritchie (1989), §5.3 e §5.5 [3]

| Linha | Problema                                                                                                                | Correção                                                                                          |
|-------|-------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|
| 2     | <b>estouro do bloco:</b> <code>malloc(n)</code> pede <i>n bytes</i> , não <i>n ints</i>                                 | <code>malloc(n * sizeof(int))</code>                                                              |
| 2     | <b>sem teste de</b> <code>NULL</code> antes de usar o bloco                                                             | <code>if (v == NULL) return NULL;</code> e testar também em <code>main</code>                     |
| 10    | <b>vazamento</b> se <code>realloc</code> falhar: <code>v</code> recebe <code>NULL</code> e o bloco antigo fica sem dono | <code>int *tmp = realloc(v, ...);</code> testar <code>tmp</code> e só então <code>v = tmp;</code> |
| 12    | <b>ponteiro pendurado:</b> lê <code>v[4]</code> depois do <code>free</code>                                             | imprimir antes; <code>free(v); v = NULL;</code> por último                                        |

## O compilador ficou quieto; o sanitizer, não

`gcc -g -fsanitize=address` para já no primeiro: `heap-buffer-overflow` na linha 4, bloco alocado na linha 2.

Fonte: King (2008), §17.1–17.4 [4]; Serebryany et al. (2012) [6]

- (a)  $i * nc + j = 2 \times 4 + 1 = 9$ .
- (b) Bloco único: **1**. `double **`:  $1 + nl = 4$   
(um para os ponteiros, um por linha).
- (c) Depois de `free(m)`, ler `m[i]` é usar memória já devolvida — ponteiro pendurado. Libere de **dentro para fora**:

```
1  for (int i = 0; i < nl; i++)  
2      free(m[i]);    /* linhas */  
3  free(m);           /* depois, m */  
4  m = NULL;
```

Fonte: Kernighan & Ritchie (1989), §5.9 [3]

- (a) **Erro de compilação:** `main.c` inclui `estat.h` duas vezes (direto e via `turma.h`) e o compilador vê `struct Resumo` definida duas vezes — `redefinition of 'Resumo'`. Correção: guarda de inclusão em todo `.h`.
- (b) Compila, mas **falha na ligação:** `turma.c` chama `estat_media`, que ninguém definiu — `undefined reference to 'estat_media'`. Faltou `estat.c` no comando.
- (c) **Falha na ligação:** `troca` está definida em dois `.o` — `multiple definition of 'troca'`. Correção: `static` nas duas (uso interno) ou uma só, declarada num `.h`.

```
#ifndef ESTAT_H
#define ESTAT_H
...           /* conteúdo do header */
#endif
```

Fonte: Kernighan & Ritchie (1989), §4.6 e §4.11.3 [3]

(a) Qualquer quatro:

- **estado global:** `n`, `k`, `R` — a entrada e a saída não aparecem na assinatura;
- **números mágicos:** 50 e 7.0;
- **nomes** de uma letra fora de laço, e `calc` sem prefixo não diz o que calcula;
- `v` só é lido: falta `const`;
- tudo público: sem `.h`, sem `static`.

(b) `R[50]` estoura: escreve além do vetor e corrompe o vizinho, sem aviso algum.

(c) Uma resposta possível:

```
#define RANKING_MEDIA_APROVACAO 7.0

/* Cópia para aprovados as medias
 * >= RANKING_MEDIA_APROVACAO.
 * aprovados tem espaço para n.
 * Devolve quantas copiou. */
int ranking_filtrar_aprovados(
    const double *medias, int n,
    double *aprovados);
```

Fonte: McConnell (2004), caps. 11 e 32 [5]

| Errou | Revise a aula                                      | Exercícios da lista |
|-------|----------------------------------------------------|---------------------|
| R1    | Fundamentos; tipos e TADs; registros e alinhamento | 1–6                 |
| R2    | Enumerações e <code>typedef</code>                 | 7, 8                |
| R3    | Ponteiros: conceitos, operadores e aritmética      | 9–12                |
| R4    | Vetores, strings e passagem por referência         | 13–16               |
| R5    | Alocação dinâmica de memória                       | 17, 18              |
| R6    | Alocação dinâmica de vetores e matrizes            | 19, 20              |
| R7    | Modularização de programas                         | slides da aula      |
| R8    | Padrões de codificação                             | slides da aula      |

## Para casa

Rodada errada: refaça no papel os exercícios indicados e releia a aula.

-  Randal E. Bryant and David R. O'Hallaron.  
*Computer Systems: A Programmer's Perspective.*  
Pearson, Boston, 3 edition, 2015.  
Seção 3.9: alinhamento de dados e layout de structs.
-  Brian W. Kernighan and Rob Pike.  
*The Practice of Programming.*  
Addison-Wesley, Reading, MA, 1999.  
Cap. 1, "Style": nomes, expressões, consistência, números mágicos e comentários.



Brian W. Kernighan and Dennis M. Ritchie.

*C: a linguagem de programação — padrão ANSI.*

Campus, Rio de Janeiro, 1989.



K. N. King.

*C Programming: A Modern Approach.*

W. W. Norton, New York, 2 edition, 2008.

Capítulo 16: estruturas, uniões e enumerações.



Steve McConnell.

*Code Complete: A Practical Handbook of Software Construction.*

Microsoft Press, Redmond, WA, 2 edition, 2004.

Cap. 11 (nomes de variáveis), cap. 31 (layout e estilo) e cap. 32 (código autodocumentado).



Konstantin Serebryany, Derek Bruening, Alexander Potapenko, and Dmitriy Vyukov.

AddressSanitizer: A fast address sanity checker.

In *USENIX Annual Technical Conference (ATC)*, pages 309–318, 2012.

Ferramenta usada no tutorial para detectar vazamentos e uso após free.